

Library Management System

Class 1: Installing Django

1. Installing Django

- Open your terminal and run the following command:

```
pip install django
```

2. Setting Up a Virtual Environment

- A virtual environment isolates your project dependencies from other projects. To set it up:

```
python -m venv myenv
```

- To activate the virtual environment:
 - **Windows:**

```
myenv\Scripts\activate
```

- **macOS/Linux:**

```
source myenv/bin/activate
```

3. Creating a Django Project

- Now, let's create a new Django project:

```
django-admin startproject library_system
```

- This creates a folder called `library_system` with all the initial files you need.

4. Understanding the Project Structure

- Inside the `library_system` folder, you'll see:
 - `manage.py`: A tool to manage the project (run server, make migrations, etc.).
 - `settings.py`: Where all your project settings go, like database configuration and installed apps.
 - `urls.py`: Contains the URL patterns (e.g., connecting pages like "Books List" to the right view).
-

Class 2: Running a Django Project

1. Running the Development Server

- Run the following command to start the development server:

```
python manage.py runserver
```

- Visit `http://127.0.0.1:8000/` in your browser to see your project live.

2. Understanding `settings.py` and `manage.py`

- **`settings.py`:** Contains configurations like database settings, installed apps, and middleware.
- **`manage.py`:** A command-line tool used to interact with your Django project.

3. Django's MVT Architecture

- **Model:** Handles the data and database. For example, in your library system, the model would define a Book.
- **View:** Handles the logic. For example, the view would decide which books to show to the user.
- **Template:** Handles the HTML structure. For example, displaying the list of books on the webpage.

4. Creating a Simple "Hello World" View

- In the `views.py` file, add this code:

```
from django.http import HttpResponse

def hello_world(request):
    return HttpResponse("Hello, World!")
```

- In `urls.py`, map this view to a URL:

```
from django.urls import path
from . import views

urlpatterns = [
    path('hello/', views.hello_world),
]
```

- Now visit `http://127.0.0.1:8000/hello/` to see "Hello, World!" displayed.
-

Class 3: Django Apps and URL Routing

1. Creating a Django App

- An app is a component of your project. In the case of the Library Management System, we'll create an app for books.

```
python manage.py startapp books
```

2. Registering Apps in `settings.py`

- Open `settings.py` and add 'books' to the `INSTALLED_APPS` list:

```
INSTALLED_APPS = [  
    'books',  # Your newly created app  
    # Other default apps  
]
```

3. URL Routing (`urls.py`)

- Now we'll set up routing to display a list of books.
 - In `urls.py`:

```
from django.urls import path  
from books import views  
  
urlpatterns = [  
    path('books/', views.book_list),  
]
```

4. Connecting Views to URLs

- In the `books/views.py` file, add the `book_list` view:

```
from django.shortcuts import render  
  
def book_list(request):  
    return render(request, 'books/book_list.html', {})
```

Class 4: Working with Views and Templates

1. Writing Views in `views.py`

- The `book_list` view will be responsible for fetching and displaying a list of books from the database (but for now, we'll just display a static list).

2. Using HTML Templates

- Create a `templates` folder in your `books` app and then create a `book_list.html` file inside `templates/books/`.
 - In `book_list.html`:

```
html
CopyEdit
<h1>Library Book List</h1>
<ul>
  <li>Book 1</li>
  <li>Book 2</li>
  <li>Book 3</li>
</ul>
```

3. Dynamic Content with Django Template Syntax

- To make the content dynamic, let's pass some data to the template. Update your view to include a list of books:

```
def book_list(request):
    books = ['Book 1', 'Book 2', 'Book 3']
    return render(request, 'books/book_list.html', {'books': books})
```

- Now, modify the `book_list.html` file to loop through the books and display them:

```
<h1>Library Book List</h1>
<ul>
  {% for book in books %}
    <li>{{ book }}</li>
  {% endfor %}
</ul>
```

4. Template Inheritance

- If you want to create a consistent design across your pages (e.g., a header or footer), you can use template inheritance.
 - Create a base template `base.html`:

```
<html>
  <body>
    <h1>{% block title %}Library System{% endblock %}</h1>
    <div>{% block content %}{% endblock %}</div>
  </body>
</html>
```

- Then, in `book_list.html`, extend this base template:

```
{% extends 'base.html' %}
{% block content %}
  <h2>Book List</h2>
  <ul>
    {% for book in books %}
```

```
        <li>{{ book }}</li>
    {% endfor %}
</ul>
{% endblock %}
```

Moving Forward:

In the upcoming classes, you'll learn how to:

- **Add books to the database:** Using Django's models to create a `Book` model.
- **Create a form:** For adding new books to the library.
- **Use Django's Admin Panel:** For easy management of books.
- **Work with Databases:** To store and retrieve real data (like book titles, authors, etc.).

This approach will help you build the **Library Management System** step by step, while learning Django in a simple and structured way.